Natural Language Processing

Tokenization | Embeddings | Transformers | spaCy | BERT

OVERVIEW

01 NLP Fundamentals

Text preprocessing, tokenization, stemming & lemmatization

02 Feature Engineering

Bag-of-Words, TF-IDF, n-grams and feature selection

03 Word Embeddings

Word2Vec, GloVe and contextual representations

04 Transformers & BERT

Attention mechanism, fine-tuning pre-trained models

05 Practical NLP Tasks

Sentiment analysis, NER, text classification

What is NLP?

Natural Language Processing (NLP) is a field of AI that enables computers to understand, interpret, and generate human language.



Understand

Parse meaning from raw text



Process

Transform & analyze language



Generate

Produce coherent human text

Real-world applications:

- → Chatbots
- → Search
- → Translation
- → Summarization
- → Sentiment

OVERVIEW · THE NLP LANDSCAPE

What Problems Does NLP Solve?



Text Classification

Spam, sentiment, topic labeling



Named Entity Recognition

Find people, places, orgs in text



Machine Translation

Convert text between languages



Text Summarization

Condense long documents



Question Answering

Extract answers from passages



Text Generation

Chatbots, autocomplete, stories

Text Preprocessing Pipeline

1. **Raw Text**
   ```
   "Hello World! NLP rocks :)"
   ```
2. **Lowercase**
   ```
   "hello world! nlp rocks :)"
   ```
3. **Remove Noise**
   ```
   "hello world nlp rocks"
   ```
4. **Tokenize**
   ```
   ["hello","world","nlp","rocks"]
   ```
5. **Lemmatize**
   ```
   ["hello","world","nlp","rock"]
   ```

Three Ways to Split Text



Word Tokenization

Input:

"playing football is fun"

Tokens:

playing	football	is	fun

✓ Simple & fast

✗ Misses morphology



Subword (BPE)

Input:

"unbelievable"

Tokens:

un	believ	able

✓ Handles rare words

✗ Less human-readable



Character Tokenization

Input:

"cat"

Tokens:

c	a	t

✓ Zero unknowns

✗ Long sequences

Tokenization with spaCy

1. Word tokenization splits text into individual tokens
2. Sentence tokenization detects sentence boundaries
3. spaCy handles punctuation & special chars
4. Each token carries POS, dependency & named-entity tags

```
import spacy

# Load model

nlp = spacy.load("en_core_web_sm")

text =  "NLP powers search engines."
doc = nlp(text)


# Token inspection

for token in doc:
   print(token.text,  token.pos_, token.lemma_)
```

FEATURE ENGINEERING

TF-IDF: Term Frequency-Inverse Document Frequency

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t)$$

TF (Term Frequency)

```
count(t in d) / total_words(d)
```

How often a term appears in a document, normalized by document length.

IDF (Inverse Document Frequency)

```
log( N / df(t) )
```

Penalizes terms that appear in many documents — rewards rare, meaningful words.

```
from sklearn.feature_extraction.text import TfidfVectorizer
vec = TfidfVectorizer(max_features=5000, ngram_range=(1,2))  # Unigrams + bigrams
X_train = vec.fit_transform(corpus)   # Sparse matrix output
```

WORD EMBEDDINGS

From One-Hot to Dense Vectors

❌ One-Hot Encoding

- ✗ Sparse: 50,000-dim vector
- ✗ No semantic meaning
- ✗ "cat" ≠ "dog" ≠ "kitten"
- ✗ Memory intensive
- ✗ Ignores context

✅ Word Embeddings

- ✓ Dense: 100-300 dims
- ✓ Captures meaning & context
- ✓ "king" − "man" + "woman" ≈ "queen"
- ✓ Efficient & reusable
- ✓ Transfer learning ready

Word2Vec (Google) | GloVe (Stanford) | FastText (Meta) | BERT (Contextual)

WORD EMBEDDINGS

Words as Points in Semantic Space

Similar meaning → close together

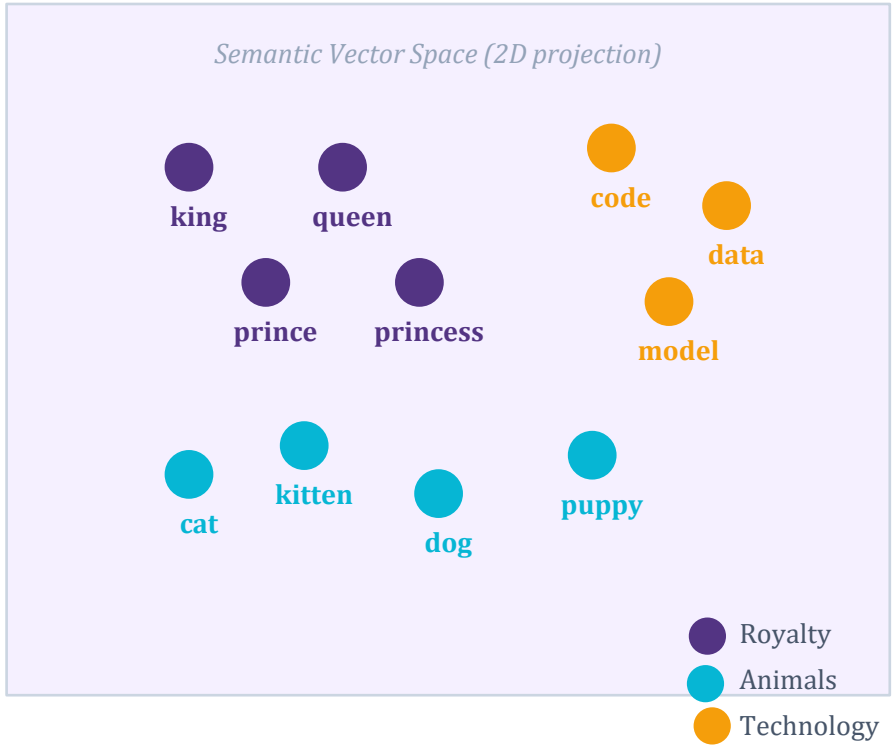
'cat' and 'kitten' cluster near each other in vector space.

Arithmetic on meaning

"king" − "man" + "woman" ≈ "queen" — geometry encodes relationships.

Transfer learning

Train once on billions of words; reuse embeddings for any task.



TRANSFORMERS & BERT

The Attention Mechanism

"Attention is All You Need" — Vaswani et al., 2017. Transformers process all tokens simultaneously and learn which words to "attend" to when encoding each position.

Q

Query

The current token asking:
'What am I looking for?'

K

Key

All tokens broadcasting:
'Here is what I contain'

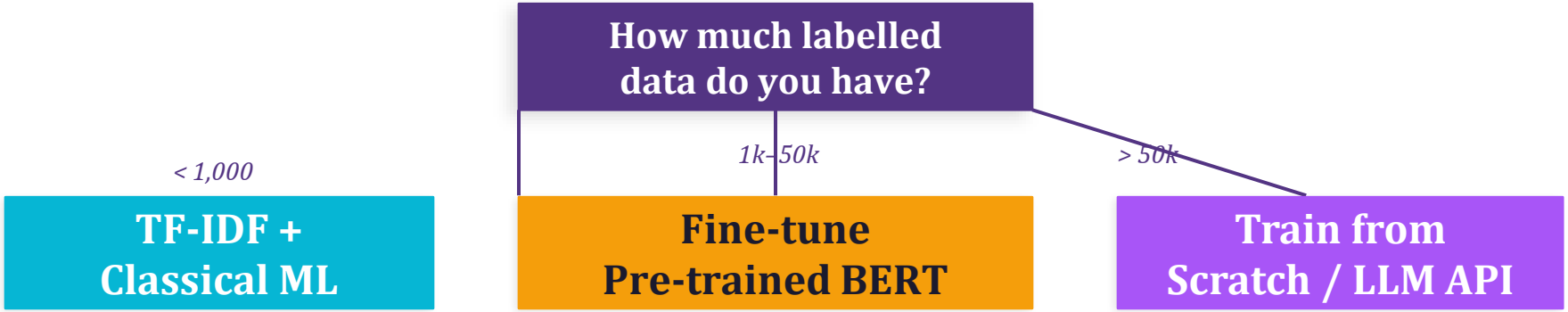
V

Value

Content to retrieve:
'If you attend to me, take this'

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k} \right) \cdot V$$

Choosing the Right NLP Approach



- ✓ Fast to train
- ✓ Interpretable
- ✓ Low compute
- ✗ No semantics
- ✗ Needs feature eng.

- ✓ Strong accuracy
- ✓ Little data needed
- ✓ Contextual
- ✗ Slower inference
- ✗ GPU recommended

- ✓ Best accuracy
- ✓ Custom vocabulary
- ✓ Scalable
- ✗ Very expensive
- ✗ Needs expert setup

NAMED ENTITY RECOGNITION

NER: Identifying Real-World Entities

PERSON
Elon Musk

ORG
OpenAI

GPE
San Francisco

DATE
January 2024

MONEY
$2.5 billion

PRODUCT
iPhone 16

```
import spacy
nlp = spacy.load("en_core_web_trf")  # Transformer model
doc = nlp("Elon Musk founded OpenAI in San Francisco.")
for ent in doc.ents:
    print(f"{ent.text:<20} → {ent.label_}"
```

EVALUATION METRICS

How to Measure NLP Model Performance

Precision	TP / (TP + FP)	Of all positive predictions, how many were correct?
Recall	TP / (TP + FN)	Of all actual positives, how many did we find?
F1 Score	2 × P × R / (P + R)	Harmonic mean of precision and recall. Best for imbalanced data.
BLEU	n-gram overlap score	For text generation & translation quality.

Colab Session

https://shorturl.at/9kIyH